

Agentic AI System for Enterprise Cloud Migrations

Timothy Wong • Alessandro Abati

London Stock Exchange Group (LSEG)

7 Oct 2026

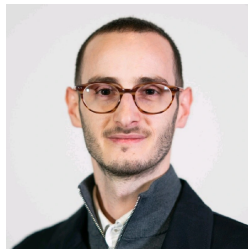


About the Speakers



Timothy Wong

Tim is an AI/ML engineer at LSEG with over ten years experience in enterprise data science applications. He hosted talks at EARL 2016, 17, 19 & 24.



Alessandro Abati

Alessandro is an AI Engineer at LSEG, specialising in agentic AI systems for enterprise applications. A newbie to EARL, but excited to share his work.



LSEG

- ▶ Leading global financial markets infrastructure and data provider.
- ▶ Operates connected businesses to serve customers across the entire financial markets value chain.
- ▶ At the heart of the world's financial ecosystem.

26K+

Employees

44K+

Customers in over
170 countries

4

Divisions – seamless access
to global financial markets

300+

Years of innovation

Our Team

We build and run LSEG's cloud foundation, services and developer experience.

1. **Business Requirements and Context**
2. **E2E Orchestration Phases**
3. **Agentic Technical Deep Dive**
 - 3.1 **Data Preprocessing**
 - 3.2 **Inside Agentic Retrieval**
 - 3.3 **Agentic Orchestration**
 - 3.4 **Agent Memory**
 - 3.5 **Agent Tools**
4. **Infrastructure & Backend**

Business Requirements and Context

Migrating enterprise applications to the cloud means **understanding them first!**

The challenge

- ▶ **Massive legacy estate:** millions of lines of code , decades in the making.
- ▶ **Scarce knowledge:** documentation is thin or outdated, and app-team experts have often left.
- ▶ **Slow discovery:** mapping dependencies and risks by hand takes weeks per app.

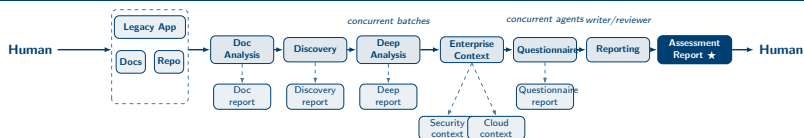
Migration patterns



From codebase to discovery report



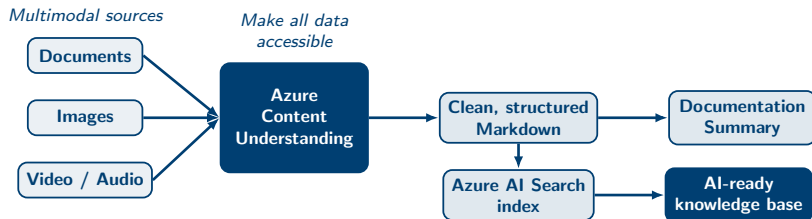
E2E Orchestration Phases



Inputs (documents & code) enter on the left; each phase records its findings and passes context forward.

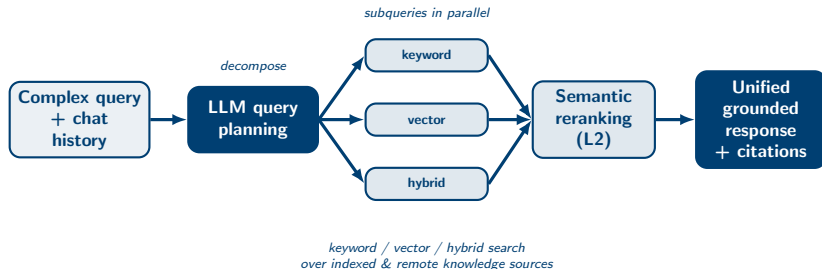
- ▶ **Discovery:** group the code and documents into logical components. *(Output: a map of the system.)*
- ▶ **Deep Analysis:** assess each component's migration readiness, in parallel. *(Output: per-component findings.)*
- ▶ **Enterprise Context:** apply organisational security and cloud standards. *(Output: policy-aligned recommendations.)*
- ▶ **Readiness Questionnaire:** answer a standard cloud-readiness questionnaire. *(Output: structured answers.)*
- ▶ **Reporting:** consolidate and review all findings. *(Output: the final assessment report.)*

Data Preprocessing - Making Documentation AI-Ready



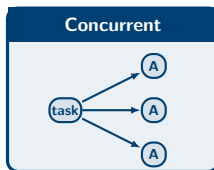
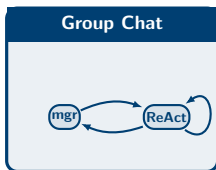
- ▶ **One analyzer, any modality:** OCR, layout, tables and figure descriptions extract meaning from otherwise unusable files.
- ▶ **Nothing lost:** charts, diagrams and scans become searchable text, not dead pixels the agents can't read.
- ▶ **Ready for retrieval:** content is chunked and embedded, so agents can query it downstream.

Inside Agentic Retrieval - A Multi-Query Pipeline



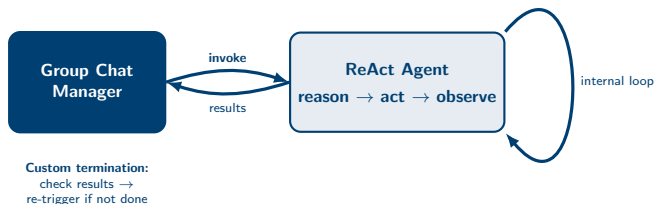
- ▶ **Beyond classic RAG:** handles multi-part questions, chat context, typos and paraphrasing that a single query can't.
- ▶ **Why decompose:** one messy question becomes several focused searches — far better recall than a single lookup.
- ▶ **Traceable:** every answer carries source citations plus an activity log agents can audit.

Agentic Orchestration - Overview



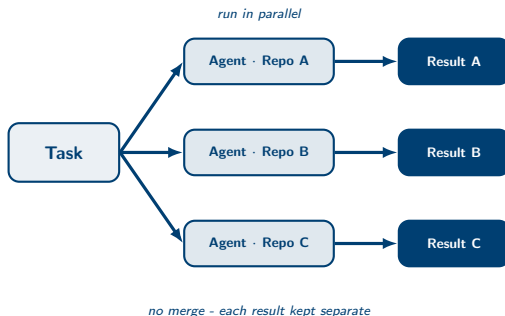
- ▶ **Framework:** built on Microsoft **Semantic Kernel**, tailored to our use case.
- ▶ **Two base patterns:** Group Chat (manager + one ReAct agent) and Concurrent (one agent per repo) — each detailed next.
- ▶ **One reused engine:** Writer/Reviewer is just Group Chat with turns taken in a fixed order.

Agentic Orchestration - Group Chat



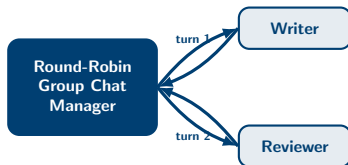
- ▶ **Not a debate:** just *one* manager over a *single* ReAct agent.
- ▶ **Why a manager:** a ReAct agent's reason-act loop is trained to stop early; the manager inspects its results and *re-triggers* it when the task isn't complete.

Agentic Orchestration - Concurrent



- ▶ **Why no merge:** each agent only needs its own repo's context, so results stay separate by design.
- ▶ **Benefits:** parallelism for speed, plus clean isolation — no cross-repo context bleed.

Agentic Orchestration - Writer/Reviewer



What Round-Robin overrides

`SelectNextAgent` - picks the next agent by a fixed **cyclic order** (a round-robin cursor).

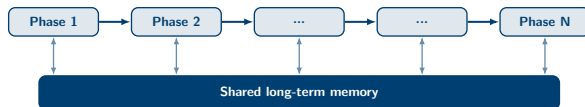
- ▶ **Deterministic turns:** Writer drafts, Reviewer critiques, alternating for a fixed number of rounds (writer gets the last word).
- ▶ **Benefits:** a built-in quality gate — iterative critique lifts output quality with no custom manager logic.

Shared long-term memory

- ▶ A single shared memory accumulates findings across the whole run.
- ▶ Each phase reads what earlier phases discovered and writes back its own results.
- ▶ Later phases build on earlier work instead of re-reading the raw inputs.

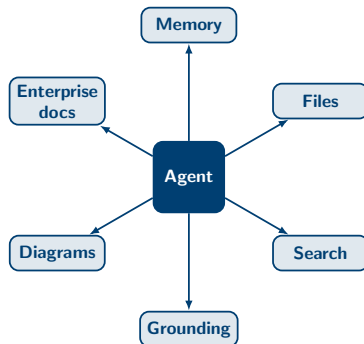
Passing context between phases

- ▶ The pipeline is serial: every phase produces a report.
- ▶ Each report, with the shared memory, becomes the input context for the next phase.
- ▶ By the final phase, agents reason over the full accumulated picture.

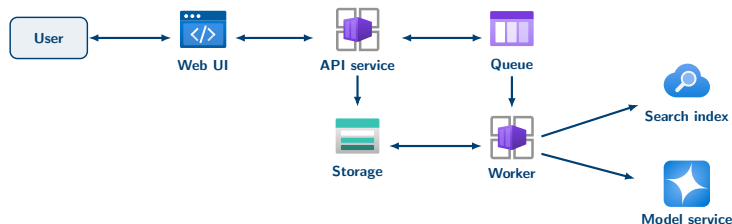


Tools available to each agent

- ▶ **Memory:** the agent keeps long-term memory as a shared file on disk, loading past findings at start-up and writing new results back after each step so knowledge persists across the run.
- ▶ **File access:** read the codebase and documents.
- ▶ **Knowledge retrieval:** ask questions over indexed documents, with cited sources.
- ▶ **Enterprise knowledge:** search organisational security, cloud and DevOps guidance.
- ▶ **Diagrams:** create, validate and fix architecture diagrams offline.



Infrastructure & Backend



- ▶ **Flow:** users upload through the web UI; the API stores inputs and queues a job; a worker then runs the phase pipeline.
- ▶ **Shared services:** the worker draws on storage, the search index and the model service to complete each phase.
- ▶ **Isolated environment:** the system runs fully offline, with automatic retries on transient failures.

Thank you
Any questions?